GhostLeaks Enterprise Edition

System Architecture

August 2026

Overview

Overview

This document explains how GhostLeaks is built and how it works internally — the request lifecycle, the data model, the breach-checking pipeline, and the security mechanisms underneath every feature. It's written for whoever will run, extend, or evaluate the codebase: your own engineers, a technical buyer doing due diligence, or you.

For *what each feature does and why*, see `FEATURE_GUIDE.md`. For *how to deploy it*, see `DEPLOYMENT.md`. This document is about *how it's built*.

---

1. Technology Stack

Layer	Choice	Why
Web framework	Flask 3.x	Small, explicit, no hidden magic — every route is a plain function you can read top to bottom.
ORM / database	Flask-SQLAlchemy 2.x, over PostgreSQL (production) or SQLite (local evaluation)	SQLAlchemy's engine abstraction means the same model code runs against either database with one connection-string change.
Auth	Flask-Login (session cookies) + a separate API-key path for the REST API	Two different consumers (a human in a browser, a machine calling the API) get two different, purpose-built auth mechanisms rather than one mechanism stretched to cover both.
Password hashing	Flask-Bcrypt	Industry-standard adaptive hashing; never reversible, never logged.
Payments	Stripe Checkout (hosted page) + webhooks	No card data ever touches this server — Stripe's hosted page handles that, and this app only ever sees a signed confirmation event.
Transactional email	Brevo (Sendinblue) API	Email verification, password reset, and breach-alert emails.
Breach intelligence	Pluggable provider modules (BreachDirectory, HIBP, LeakCheck, XposedOrNot)	See §5 — no single vendor is hardcoded into the application logic.
PDF generation	ReportLab	Server-side rendering of tenant-branded exposure reports.

Rate limiting	Flask-Limiter, in-memory by default, Redis-backed if `REDIS_URL` is set	See §7.
WSGI server	Gunicorn	Standard production entry point (`gunicorn app:app`).

The application is a single Flask app (`app.py`) — there is no microservice split, no message queue, no background worker process. Every request is handled synchronously, start to finish, by a Gunicorn worker. This is a deliberate simplicity choice: at the scale a single self-hosted deployment (or even a multi-tenant MSP deployment with dozens of client workspaces) actually runs at, a job queue adds operational complexity without a corresponding benefit. See §9 for where this stops being true.

---

2. Request Lifecycle

A typical browser request — say, a logged-in user running an email breach check — flows like this:

1. **Gunicorn** accepts the HTTP request and hands it to a Flask worker process.
2. **Flask-Login** resolves the session cookie to a `User` row (`load_user()`), making `current_user` available for the rest of the request. If there's no valid session, `current_user` is an anonymous user and any `@login_required` route redirects to `/login`.
3. **Flask-Limiter** checks the request against the route's configured rate limit (e.g. `"30 per minute"` on `/check-email`) before the view function runs at all. Over the limit → an immediate `429` with `Retry-After` headers, no application logic executes.
4. **CSRF protection** (Flask-WTF) validates the submitted token against the session for any state-changing `POST` request that isn't explicitly exempted (the two API-key-authenticated JSON endpoints are exempted, since they don't use session cookies in the first place — see §6).
5. The **view function** runs: reads `request.form` / `request.get_json()`, does its work, and returns a response.
6. `@app.after_request` attaches baseline security headers (CSP, `X-Frame-Options`, HSTS, etc.) to every response before it goes back to the browser — this runs regardless of which route handled the request, so no route can accidentally ship without them.

For an API request (`X-API-Key` header instead of a session cookie), steps 2–4 are replaced by `require_api_key()`: it hashes the provided key, looks up the matching `User` by that hash, and attaches it to `request.api_user` — the rest of the request proceeds identically.

---

3. Data Model

Six tables, all scoped to a tenant except where noted:

- `Tenant` — one row per workspace/client company. Holds branding (`brand_name`, `primary_color`, `logo_url`), plan, and demo-mode state. This is the isolation boundary everything else hangs off of.
- `User` — belongs to exactly one tenant, has a `role` (`owner` / `admin` / `member`), a bcrypt password hash, an API key (stored as a SHA-256 hash — see §6), payment status, and webhook URL.
- `ScanHistory` — one row per breach check performed (email, password, or bulk), what it found, and when. This is *usage* data — "what was scanned."
- `CredentialFinding` — one row per (tenant, email, breach source) combination, deduplicated and upserted on every re-scan rather than duplicated. This is *state* data — "what's currently true about this exposure," carrying a status lifecycle (`active` → `rotated` / `historical` / `accepted_risk` / `resolved`) and a confidence score.
- `DomainMonitor` + `DomainEmployeeEmail` — a company domain a tenant wants monitored as a whole, gated behind DNS-based ownership verification (§4), with individual employee emails tracked underneath it.
- `AuditLog` — administrative and state-changing actions (logins, status changes, domain verification, branding changes, API key rotation, account deletion). Distinct from `ScanHistory`: this answers "who changed what," not "what was scanned."

Every table except `Tenant` itself carries a `tenant_id` foreign key, and every query in the codebase filters on it. This is enforced by code convention, not by a database-level policy (no Postgres row-level security) — see §9 for the implication of that.

---

4. Multi-Tenancy and Isolation

A tenant is created one of two ways: - **Register with no invite link** → a brand-new `Tenant` is created, and the registering user becomes its `owner`. - **Register with `?tenant_slug=...`** (an admin's invite link) → joins that existing tenant as a `member`.

From that point forward, every database query touching tenant-scoped data filters on `current_user.tenant_id` (or `request.api_user.tenant_id` for API calls). There is no code path anywhere in the application — UI or API — that returns data belonging to a different tenant. This is the single rule the whole multi-tenant model rests on, and it's why the product can safely serve multiple unrelated client companies from one deployment.

---

5. The Breach-Checking Pipeline

This is the core of the product, and it's built to be provider-agnostic on purpose.

Provider architecture: each breach-data source (BreachDirectory, HIBP, LeakCheck, XposedOrNot as shipped) is a small module in `providers/`, implementing one shared interface defined in `providers/base_provider.py`. Which provider(s) are active is controlled entirely by the `BREACH_PROVIDER` environment variable — a single name, or a comma-separated list to run several at once. Adding a new data source later means writing one new provider module and adding one line to the factory function in `app.py`; nothing else in the application needs to change.

What actually happens on a check: 1. `aggregate_breach_sources(email)` calls every currently-active provider. 2. Each provider returns a list of source names it found the email in, or signals it couldn't respond (`check_failed`). 3. Results are tallied by source name across all providers — if two providers both report "LinkedIn," that's one finding with `confirmed_by = 2`, not two separate findings. 4. `compute_confidence()` scores the finding: 60 as a base for a single-source hit, +20 per additional confirming source (capped at +40), +5 if the exposure has been outstanding over a year. With only one provider configured, no finding can ever be cross-confirmed, so everything caps at 60–65 — that's the pipeline being honest about single-source confidence, not a bug. 5. The result is upserted into `CredentialFinding`, keyed on `(tenant_id, email, source_name)` — re-scanning the same email against the same source updates the existing row (`last_seen`, confidence) instead of creating a duplicate, so status history (§3) survives repeat scans.

A hard rule that runs through every one of these code paths: the actual leaked password or credential value is never read out of a provider's response, never stored, and never returned to a user — only the fact of exposure (which source, roughly when, how confident). Provider modules are written to not even parse that field out of the response. This is what makes the product a breach-*notification* tool rather than a credential-lookup tool, and it's treated as non-negotiable — see `SECURITY.md`.

Domain-wide monitoring (§ in `FEATURE_GUIDE.md`) runs the same pipeline once per tracked employee email, rather than trying to check a bare domain string — breach providers index individual addresses, not domains, so that's what actually finds real exposures. It's gated behind a live DNS TXT-record check before any domain-wide scan is allowed to run, so a tenant can't enumerate exposure data for a domain it doesn't control.

---

6. Authentication and API Security

Session-based (browser) auth: standard Flask-Login session cookies, hardened with `HttpOnly`, `SameSite=Lax`, and `Secure` (over HTTPS). CSRF tokens are required on every state-changing form

submission.

API-key auth: a key is generated the first time a user visits Settings (`gl_` prefix + 48 hex characters), and can be regenerated at any time, immediately invalidating the old one. Keys are stored as a SHA-256 hash — the raw value is shown exactly once, at generation time, and is never recoverable from the database afterward. The two REST endpoints that use this (`/api/v1/check-email` , `/api/v1/check-password`) are explicitly exempted from CSRF checks, since CSRF protection exists to defend session-cookie auth specifically — a request authenticated by a header the browser doesn't attach automatically isn't vulnerable to the cross-site form-submission attack CSRF tokens defend against.

Outbound webhooks: before every send (not just when the URL is first saved), the destination hostname is resolved and checked against private/loopback/link-local/reserved IP ranges, and the outbound request has redirects disabled. Both exist to close a specific gap: a hostname can resolve to a public IP when saved and a private one later (DNS rebinding), and `requests` follows redirects by default, which would otherwise let an allowed `https://` host 30x to an internal address.

Payment webhook: Stripe webhook payloads are verified against their signature (using `STRIPE_WEBHOOK_SECRET`) before being trusted — an unsigned or incorrectly-signed request is rejected outright, so activation can't be triggered by anyone who doesn't hold Stripe's actual signing secret.

Scheduled-job auth: the breach-alert cron endpoint (`/send-breach-alerts`) is protected by a shared secret compared using `hmac.compare_digest` (constant-time, to avoid timing attacks) and passed via a custom header rather than a query parameter — query parameters end up in server logs and browser history, a header does not.

---

7. Rate Limiting

Flask-Limiter enforces per-route limits (e.g. 10 registrations/hour, 30 checks/minute, 3 bulk scans/day) to prevent abuse and control third-party provider API spend. By default this state lives in-memory inside each Gunicorn worker process — which means the *effective* limit is roughly the configured value multiplied by however many workers are running, and resets whenever a worker restarts. Setting `REDIS_URL` switches to a shared Redis-backed store, giving accurate limits across every worker and every restart. For a single-worker deployment this distinction doesn't matter; for anything scaled beyond one worker, Redis is the correct configuration, not an optional upgrade.

---

8. Payment Architecture

Stripe Checkout is used as a **hosted page**, not a custom card form — the browser is redirected to a page Stripe controls, and this application never sees, handles, or stores any card data. Two things happen on the way there and back:

- **`/create-checkout-session`** creates a Stripe Checkout Session tied to `current_user` , recording which order belongs to which user before redirecting.
- **`/webhooks/stripe`** receives Stripe's `checkout.session.completed` event, verifies its signature, and only then marks the user `is_paid` .

Activation is driven entirely by the webhook, not by a client-side "payment succeeded" callback — a webhook still fires even if the customer's browser closes or their network drops right after paying, and a webhook can't be forged into activating an account that didn't actually pay, the way a client-side callback could be. Price and currency are read from environment variables (`PLAN_AMOUNT_CENTS` , `PLAN_CURRENCY` , `PLAN_PRICE_LABEL`) — nothing is hardcoded, so a deployment can price itself in any Stripe-supported currency without a code change.

---

9. Known Architectural Boundaries

Every system has edges. These are the ones worth knowing before you extend or scale this one:

- **No database migration tool.** Schema changes rely on `db.create_all()`, which creates missing *tables* but never adds a missing *column* to a table that already has data in it. A fresh deployment is unaffected; an existing production database needs the exact `ALTER TABLE` statement run manually for each schema change (documented per-change in `DEPLOYMENT.md`). Adopting a migration tool (e.g. Flask-Migrate/Alembic) is the natural next step if the schema will keep evolving.
- **No database-level tenant isolation.** Isolation is enforced by every query filtering on `tenant_id` in application code, not by a Postgres row-level security policy. For most deployments this is sufficient; if you're serving mutually distrustful tenants at a scale where a single missed filter would be catastrophic, RLS is a reasonable second line of defense to add.
- **No background job queue.** Bulk CSV scans and the scheduled breach-alert job run synchronously, inside the request or cron-triggered request itself. This is fine at the scale currently tested; a very large bulk upload or a very large user base on the scheduled job could approach a request timeout before a queue-based redesign becomes necessary.
- **Rate-limit storage defaults to in-memory** (see §7) — correct for a single worker, needs `REDIS_URL` beyond that.
- **No automated test suite.** Changes are verified by static syntax/import checks and manual testing, not a CI test suite exercising application behavior.

None of these are defects — they're the tradeoffs of a codebase sized correctly for what it actually needs to do today, documented honestly so whoever operates or extends it knows exactly where the edges are.